

Practical No.5

Aim: Implement File Management Tasks in Sqoop over Hadoop.

DESCRIPTION:

This practical demonstrates creating a database, table, inserting records in MySQL, and importing them into Hadoop HDFS using Sqoop on HDP Sandbox.

Step 1: Connect to HDP Sandbox via PuTTY

1. Open PuTTY.
2. Hostname: 127.0.0.1
3. Port: 2222 (for SSH into sandbox)
4. Username: root
5. Password: hadoop (default).



```
root@sandbox-hdp:~#
login as: root
root@127.0.0.1's password:
You are required to change your password immediately (root enforced)
Last login: Mon Jun 18 15:28:54 2018 from 172.17.0.2
Changing password for root.
(current) UNIX password:
New password:
BAD PASSWORD: The password is the same as the old one
New password:
Retype new password:
[root@sandbox-hdp ~]#
```

Step 2: Login to MySQL mysql

-u root -p

(Enter password: hadoop)

```
root@sandbox-hdp:~# mysql
mysql> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.00 sec)

mysql> QUIT;
Bye
[root@sandbox-hdp ~]# systemctl unset-environment MYSQLD_OPTS
[root@sandbox-hdp ~]# systemctl restart mysqld
[root@sandbox-hdp ~]# exit
exit
[root@sandbox-hdp ~]# mysql -uroot -phadoop
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.7.22 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Step 3: Create Database

CREATE DATABASE company;

USE company;

```
root@sandbox-hdp:~# mysql
[root@sandbox-hdp ~]# systemctl unset-environment MYSQLD_OPTS
[root@sandbox-hdp ~]# systemctl restart mysqld
[root@sandbox-hdp ~]# exit
exit
[root@sandbox-hdp ~]# mysql -uroot -phadoop
mysql: [Warning] Using a password on the command line interface can be insecure.
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 7
Server version: 5.7.22 MySQL Community Server (GPL)

Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE company;
Query OK, 1 row affected (0.01 sec)

mysql> USE company;
Database changed
mysql>
```

Step 4: Create Table

```
CREATE TABLE employee ( id INT PRIMARY
KEY AUTO_INCREMENT,
name          VARCHAR(50),
department VARCHAR(50),
salary INT
);
```

```
root@sandbox-hdp:~  
[root@sandbox-hdp ~]# mysql -uroot -phadoop  
mysql: [Warning] Using a password on the command line interface can be insecure.  
Welcome to the MySQL monitor.  Commands end with ; or \g.  
Your MySQL connection id is 7  
Server version: 5.7.22 MySQL Community Server (GPL)  
  
Copyright (c) 2000, 2018, Oracle and/or its affiliates. All rights reserved.  
  
Oracle is a registered trademark of Oracle Corporation and/or its  
affiliates. Other names may be trademarks of their respective  
owners.  
  
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.  
  
mysql> CREATE DATABASE company;  
Query OK, 1 row affected (0.01 sec)  
  
mysql> USE company;  
Database changed  
mysql> CREATE TABLE employee(id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50)  
, department VARCHAR(50), salary INT);  
Query OK, 0 rows affected (0.25 sec)  
  
mysql>
```

Step 5: Insert Records

INSERT INTO employee (name, department, salary) VALUES

('Pratik', 'BigData', 45000),

('Anita', 'Analytics', 50000),

('Rohan', 'DevOps', 55000);

Verify

SELECT * FROM employee;

exit;

```
root@sandbox-hdp:~  
Query OK, 1 row affected (0.01 sec)  
  
mysql> USE company;  
Database changed  
mysql> CREATE TABLE employee(id INT PRIMARY KEY AUTO_INCREMENT, name VARCHAR(50)  
, department VARCHAR(50), salary INT);  
Query OK, 0 rows affected (0.25 sec)  
  
mysql> INSERT INTO employee (name, department, salary) VALUES('Pratik','BigData'  
, 45000), ('Anita','Analytics',50000), ('Rohan','DevOps',55000);  
Query OK, 3 rows affected (0.05 sec)  
Records: 3 Duplicates: 0 Warnings: 0  
  
mysql> SELECT * FROM employee;  
+----+-----+-----+-----+  
id | name  | department | salary |  
+----+-----+-----+-----+  
1  | Pratik | BigData   | 45000 |  
2  | Anita  | Analytics | 50000 |  
3  | Rohan  | DevOps    | 55000 |  
+----+-----+-----+-----+  
3 rows in set (0.00 sec)  
  
mysql>
```

Step 6: Import Table into HDFS using Sqoop sqoop

import \

--connect jdbc:mysql://localhost/company?useSSL=false \

```
--username root \
--password hadoop \
--table employee \
-m 1 \
--target-dir /user/root/employee_data
```

```
root@sandbox-hdp:~#
reduce
Note: /tmp/sqoop-root/compile/2c5ec7b6d135135478234c2956c89490/employee.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
25/09/22 09:02:25 INFO oom.CompilationManager: Writing jar file: /tmp/sqoop-root/compile/2c5ec7b6d135135478234c2956c89490/employee.jar
25/09/22 09:02:25 WARN manager.MySQLManager: It looks like you are importing from mysql.
25/09/22 09:02:25 WARN manager.MySQLManager: This transfer can be faster! Use the --direct
25/09/22 09:02:25 WARN manager.MySQLManager: option to exercise a MySQL-specific fast path.
25/09/22 09:02:25 INFO manager.MySQLManager: Setting zero DATETIME behavior to convertToNull (mysql)
25/09/22 09:02:26 INFO mapreduce.ImportJobBase: Beginning import of employee
25/09/22 09:02:37 INFO client.RMProxy: Connecting to ResourceManager at sandbox-hdp.hortonworks.com/172.18.0.2:8032
25/09/22 09:02:38 INFO client.AHSProxy: Connecting to Application History server at sandbox-hdp.hortonworks.com/172.18.0.2:10200
25/09/22 09:03:15 INFO db.DBInputFormat: Using read committed transaction isolation
25/09/22 09:03:17 INFO mapreduce.JobSubmitter: number of splits:1
25/09/22 09:03:18 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1758530426240_0001
25/09/22 09:03:21 INFO impl.YarnClientImpl: Submitted application application_1758530426240_0001
25/09/22 09:03:21 INFO mapreduce.Job: The url to track the job: http://sandbox-hdp.hortonworks.com:8088/proxy/application_1758530426240_0001/
25/09/22 09:03:21 INFO mapreduce.Job: Running job: job_1758530426240_0001
^C[root@sandbox-hdp ~]#
```

Step 7: Verify Data in HDFS `hdfs dfs`

```
-ls /user/root/employee_data
```

```
hdfs dfs -cat /user/root/employee_data/part-m-00000
```

```
root@sandbox-hdp:~#
[root@sandbox-hdp ~]# hdfs dfs -ls /user/root/employee_data
Found 2 items
-rw-r--r-- 1 root hdfs 0 2025-09-22 09:12 /user/root/employee_data/SUCCESS
-rw-r--r-- 1 root hdfs 68 2025-09-22 09:12 /user/root/employee_data/part-m-00000
[root@sandbox-hdp ~]# hdfs dfs -cat /user/root/employee_data/part-m-00000
1,Pratik,BigData,45000
2,Anita,Analytics,50000
3,Rohan,DevOps,55000
[root@sandbox-hdp ~]#
```

Step 8 (Optional): Import into Hive `sqoop`

```
import \
```

```
--connect jdbc:mysql://localhost/company?useSSL=false \
```

```
--username root \
```

```
--password hadoop \
```

```
--table employee \
```

```
--hive-import \
```

```
--create-hive-table \
```

```
--hive-table company_hive.employee_hive \ -
```

```
m 1
```

```
# Verify in Hive hive
```

```
USE company_hive;
```

```
SELECT * FROM employee_hive;
```

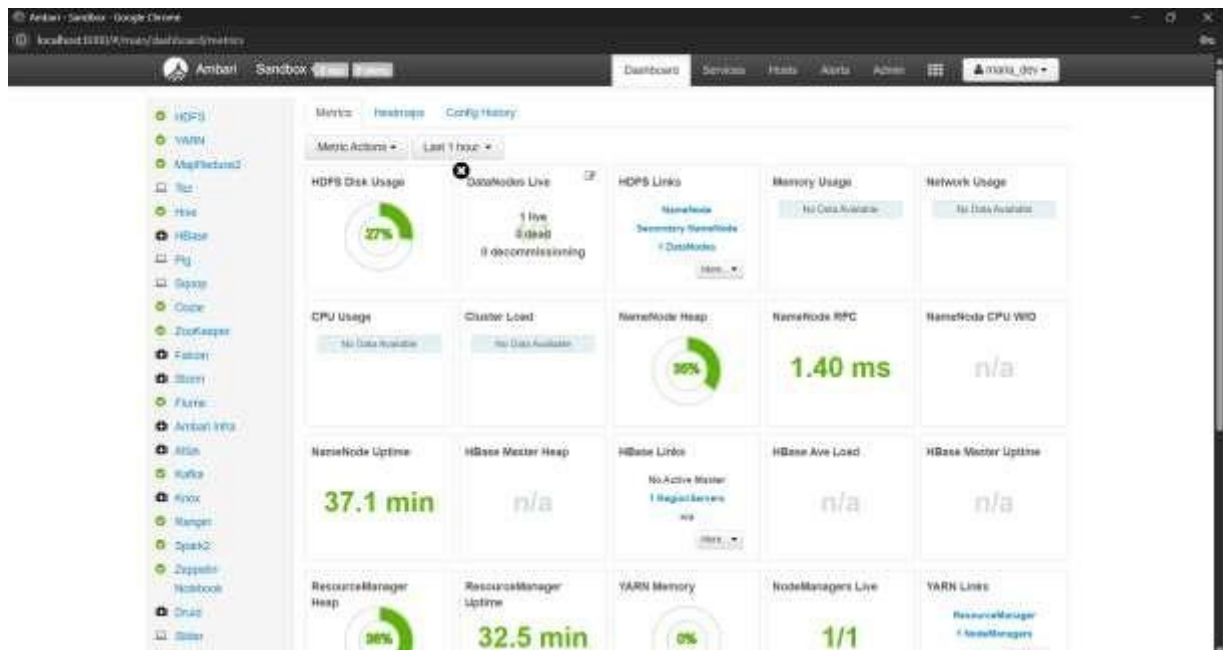
```
root@sandbox-hdp:~  
[java:640]  
    at org.apache.hadoop.ipc.RPC$Server.call(RPC.java:982)  
    at org.apache.hadoop.ipc.Server$Handler$1.run(Server.java:2351)  
    at org.apache.hadoop.ipc.Server$Handler$1.run(Server.java:2347)  
    at java.security.AccessController.doPrivileged(Native Method)  
    at javax.security.auth.Subject.doAs(Subject.java:422)  
    at org.apache.hadoop.security.UserGroupInformation.doAs(UserGroupInformation.java:1869)  
    at org.apache.hadoop.ipc.Server$Handler.run(Server.java:2347)  
  
FAILED: Execution Error, return code 1 from org.apache.hadoop.hive.q1.exec.MoveTask  
[root@sandbox-hdp ~]# hive  
log4j:WARN No such property [maxFileSize] in org.apache.log4j.DailyRollingFileAppender.  
  
Logging initialized using configuration in file:/etc/hive/2.6.5.0-292.0/hive-log4j.properties  
hive> USE company_hive;  
OK  
Time taken: 2.391 seconds  
hive> SELECT * FROM employee_hive;  
OK  
1      Pratik  BigData 45000  
2      Anita   Analytics 50000  
3      Rohan   DevOps 55000  
Time taken: 1.011 seconds, Fetched: 3 row(s)  
hive> █
```

Practical No.6

Aim: To create and upload tables in Hive using Ambari Hive View and execute SQL queries to analyze movie rating data.

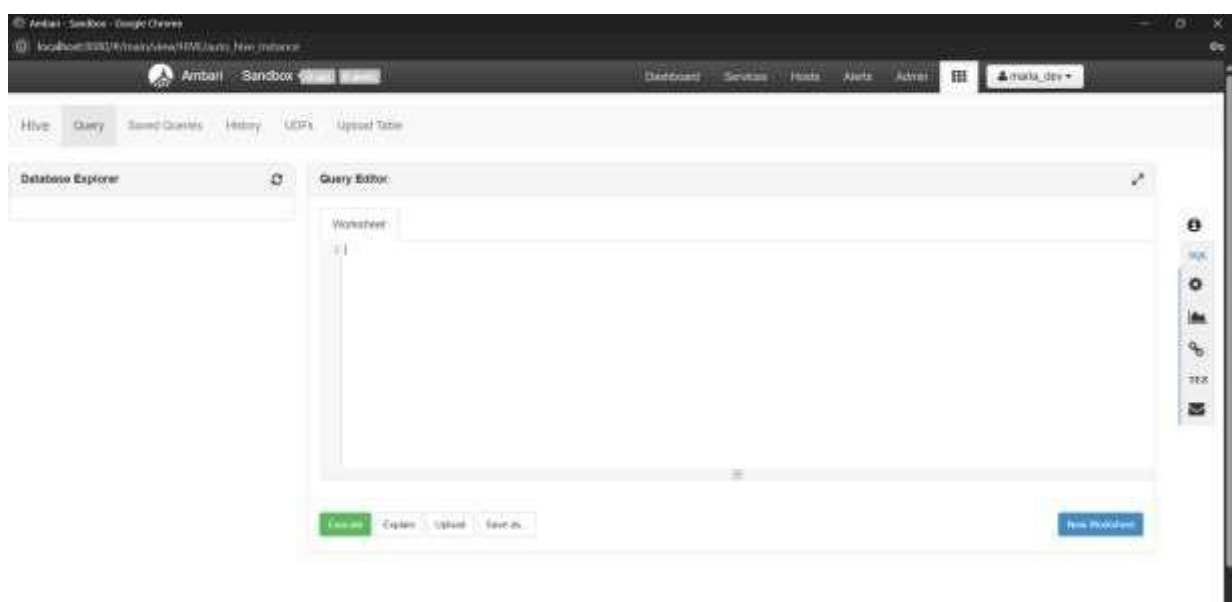
Step 1: Login to Ambari

Open web browser and enter the Ambari URL (e.g., <http://sandbox-hdp.hortonworks.com:8080>). Login with credentials: Username: maria_dev, Password: maria_dev.



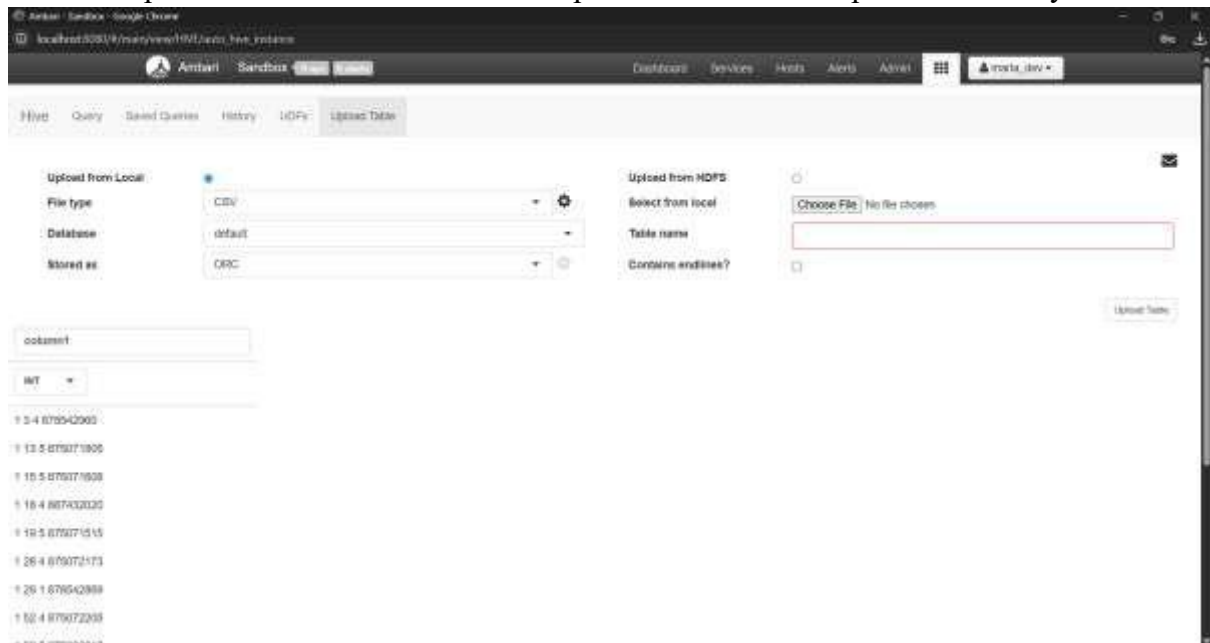
Step 2: Open Hive View

From the Ambari dashboard, click on the 'View' icon and select 'Hive View' to open the Hive interface.



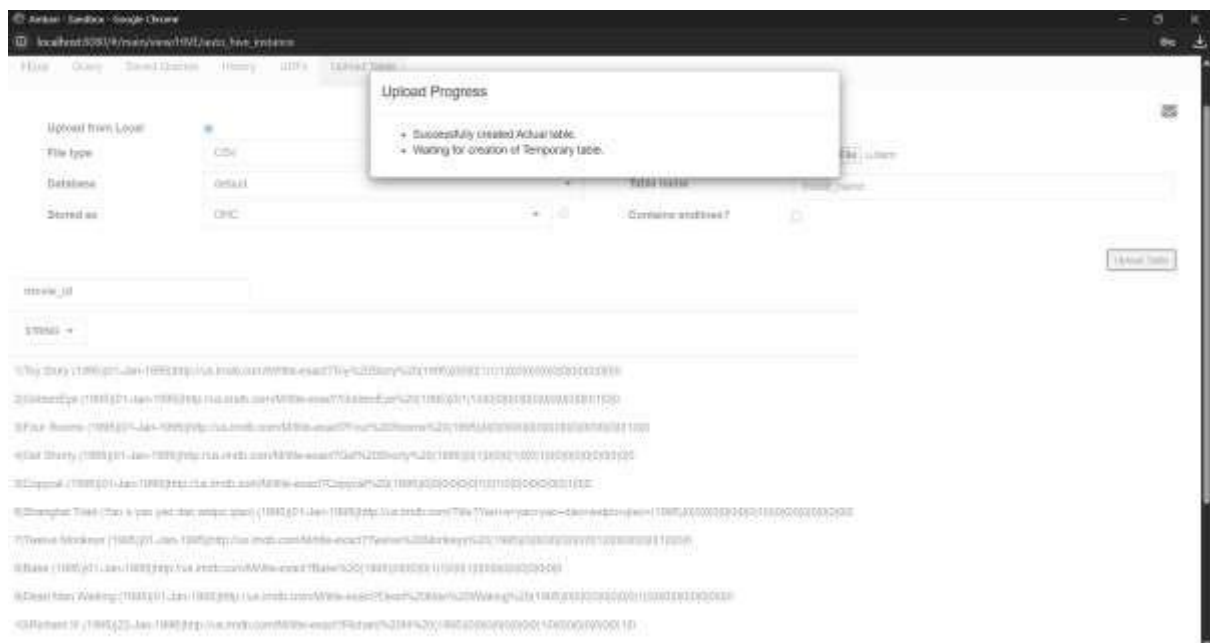
Step 3: Upload a Table

Click on 'Upload Table' and follow the same process as in the first practical activity.



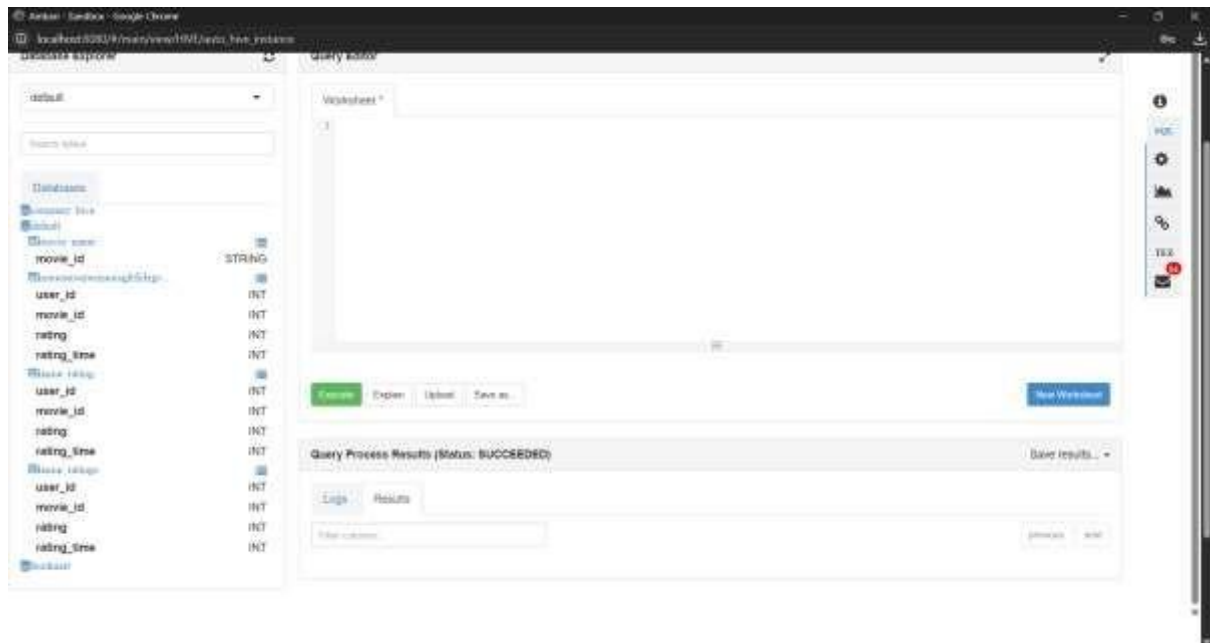
Step 4: Provide Table Details

Enter the table name and column names as in the previous practical, and upload the files u.data and u.item.



Step 5: Verify Upload

Ensure both tables (u.data and u.item) are successfully uploaded and visible in the Hive Tables list.



Step 6: Write and Execute SQL Queries

In the Query Editor, write the following SQL queries as an example:

SQL Queries:

```
CREATE VIEW topMovieIDs AS
SELECT movieID, COUNT(movieID) AS ratingCount
FROM ratings
GROUP BY movieID
ORDER BY ratingCount DESC;
SELECT n.title, t.ratingCount
FROM topMovieIDs t
JOIN names n
ON t.movieID = n.movieID;
```

Explanation: The first query creates a view showing movie IDs with the highest rating counts. The second query joins this view with the names table to display movie titles with their rating count.

Step 7: Run the Query

Click 'Execute' or 'Run' to run the queries and observe the output in the Results panel.

mn.name	t.ratingcount
Star Wars (1977)	583
Contact (1997)	509
Fargo (1996)	508
Return of the Jedi (1983)	507
Liar Liar (1997)	485
English Patient, The (1996)	481
Scream (1996)	478
Toy Story (1995)	452
Air Force One (1997)	431
Independence Day (ID4) (1996)	429
Raiders of the Lost Ark (1981)	420
Godfather, The (1972)	413
Pulp Fiction (1994)	394
Twelve Monkeys (1995)	392
Silence of the Lambs, The (1991)	390
Jerry Maguire (1996)	384
Rock, The (1996)	378
Empire Strikes Back, The (1980)	367
Star Trek: First Contact (1996)	365
Back to the Future (1985)	350
Titanic (1997)	350

Result:

The SQL queries execute successfully and display the list of movies with the highest number of ratings.

Conclusion:

Using Ambari Hive View, data can be uploaded easily into Hive tables, and SQL operations such as creating views, aggregations, and joins can be performed efficiently without using the command-line interface.

Practical No.7

Aim: Install and Run Pig then write Pig Latin scripts to sort, group, join, project and filter the data.

DESCRIPTION:

This practical demonstrates how to use the Ambari Hive View as a graphical user interface for data management and analysis in Hive. It involves uploading movie dataset files to create Hive tables and then running SQL queries to create views, perform aggregations, and join tables to find the movies with the highest number of ratings. The entire process is completed without using the command-line interface.

STEPS OF EXECUTION:

1. Login to Ambari: Open a web browser and navigate to the Ambari URL, then log in with the username `maria_dev` and password `maria_dev`.
2. Navigate to Hive View: From the main Ambari dashboard, click the 'View' icon and select 'Hive View' to access the Hive query interface.
3. Upload Data Tables:
 - a. Click on '**Upload Table**' to begin the process.
 - b. Provide the necessary table details, including table and column names.
 - c. Upload the **u.data** and **u.item** files to create two separate tables.
 - d. Verify that both tables are now visible in the Hive Tables list.
4. Write and Execute Queries: In the Query Editor, enter the following SQL queries.
 - a. First, create a view that counts the number of ratings for each movie and orders them from highest to lowest. **CREATE VIEW topMovieIDs AS
SELECT movieID, COUNT(movieID) AS ratingCount
FROM ratings
GROUP BY movieID
ORDER BY ratingCount DESC;**
 - b. Next, join this view with a names table to get the movie titles corresponding to the IDs.
**SELECT n.title, t.ratingCount
FROM topMovieIDs t JOIN
names n
ON t.movieID = n.movieID**
5. **View Results:** Click the '**Execute**' button to run the queries. The results, showing a list of movies with the highest number of ratings, will be displayed in the Results panel.



Movie ID	Name	Release Date	Release	Created
1	Toy Story (1995)	25-Jan-1995	http://www.imdb.com/AM/AM.navi?Toy%20Story%20%201995	
2	Indiana Jones (1981)	25-Jan-1981	http://www.imdb.com/AM/AM.navi?Indiana%20Jones%20%201981	
3	Paul Robeson (1985)	25-Jan-1985	http://www.imdb.com/AM/AM.navi?Paul%20Robeson%20%201985	
4	Get Shorty (1995)	25-Jan-1995	http://www.imdb.com/AM/AM.navi?Get%20Shorty%20%201995	
5	Chicago (1985)	25-Jan-1985	http://www.imdb.com/AM/AM.navi?Chicago%20%201985	
6	Shogun: The Last of the Samurai (1985)	25-Jan-1985	http://www.imdb.com/AM/AM.navi?Shogun%20The%20Last%20of%20the%20Samurai%20%201985	
7	Twelve Monkeys (1995)	25-Jan-1995	http://www.imdb.com/AM/AM.navi?Twelve%20Monkeys%20%201995	

```

1 -- Drop the view if it already exists to avoid errors
2 DROP VIEW IF EXISTS topMovieIDs;
3
4 -- Create the view by counting ratings for each movie
5 CREATE VIEW topMovieIDs AS
6 SELECT
7     movie_id,
8     COUNT(movie_id) AS ratingCount
9 FROM
10 ratings
11 GROUP BY
12     movie_id;

```

```

1 -- Now, select from the view you just created
2 SELECT
3     mn.name,
4     t.ratingCount
5 FROM
6     topMovieIDs t
7 JOIN
8     movie_name mn ON t.movie_id = mn.movie_id
9 ORDER BY
10     t.ratingCount DESC;

```

mn.name	t.ratingcount
Star Wars (1977)	583
Contact (1997)	509
Fargo (1996)	508
Return of the Jedi (1983)	507
Liar Liar (1997)	485
English Patient, The (1996)	481
Scream (1996)	478
Toy Story (1995)	452
Air Force One (1997)	431
Independence Day (ID4) (1996)	429
Raiders of the Lost Ark (1981)	420
Godfather, The (1972)	413
Pulp Fiction (1994)	394
Twelve Monkeys (1995)	392
Silence of the Lambs, The (1991)	390
Jerry Maguire (1996)	384
Rock, The (1996)	378
Empire Strikes Back, The (1980)	367
Star Trek: First Contact (1996)	365
Back to the Future (1985)	350
Titanic (1997)	350

CONCLUSION:

Using the Ambari Hive View, data can be uploaded into Hive tables easily. This interface allows for efficient execution of SQL operations, including creating views, performing aggregations, and joining tables, all without needing to use the commandline interface.